

My Agentic System Design Journey

The 13 pillars a production-grade agentic AI system actually needs — no filler, just what's required to make it real, safe, and reliable.

✓ All 13 Steps — Full Roadmap

Notes written in simple words, the way I understood them.

- ➔ Fundamentals
- ➔ Agent Harness / Runtime
- ➔ Clear Model Routing
- ➔ Strict Tool Contracts
- ➔ Explicit Memory & State Management
- ➔ Orchestration
- ➔ Trace-Level Evals
- ➔ Approval Gates
- ➔ Cost & Latency Controls
- ➔ Security
- ➔ Privacy
- ➔ Content Management & Output Safety
- ➔ Deployment & Reliability

STEP 1

Fundamentals

The mental model everything else builds on

1	What is an agent? Agent vs chatbot vs a single LLM call
2	The agent loop: think → act → observe → repeat
3	Agent patterns: ReAct, Plan-and-Execute, computer-use agents, coding agents
4	Planning & reasoning: chain of thought, task decomposition, reflection

Step 1 of 13

STEP 2

Agent Harness / Runtime

The engine that actually runs the agent loop

1	What is a "harness"? The code that assembles prompts, calls tools, and re-feeds results
2	Harness responsibilities: prompt assembly, tool execution, output parsing, loop control
3	Real examples: Claude Code's harness, OpenAI Assistants runtime, custom harnesses
4	Stopping conditions: max iterations, budget caps, success criteria, timeouts

Step 2 of 13

STEP 3

Clear Model Routing

Not every task needs your biggest, most expensive model

1	Why production systems don't use one model for everything
2	Cheap/fast model vs expensive/smart model, routed by task complexity
3	Cost vs latency vs quality tradeoffs

Step 3 of 13

STEP 4

Strict Tool Contracts

How agents actually DO things, reliably

1	Function/tool calling and designing a clean tool schema
2	MCP (Model Context Protocol) — the standard way to plug tools & data into any agent
3	Structured output & schema validation (e.g. Pydantic) so output is always usable
4	Validating tool arguments before executing — catching hallucinated or malformed calls

Step 4 of 13

STEP 5

Explicit Memory & State Management

Knowing what the agent remembers, and never losing its place

1	Short-term memory (context window) vs long-term memory (vector DBs, embeddings)
2	Context engineering: what tokens go in, budget management, compaction
3	Durable execution — resuming a task after a crash or restart, not starting over
4	Checkpointing state after every step

Step 5 of 13

STEP 6

Orchestration

Coordinating more than one agent or step cleanly

1	Orchestrator-worker pattern, subagents, and task handoff
2	Agent-to-agent communication (A2A)
3	Orchestration frameworks: LangGraph, CrewAI, AutoGen — what each solves
4	When multi-agent actually helps vs just adds complexity

Step 6 of 13

STEP 7

Trace-Level Evals

How do you actually know if an agent is good?

1	Golden datasets and eval sets
2	Trajectory evaluation — did it take the right steps, not just reach the right answer
3	LLM-as-judge and human evaluation
4	Observability & tracing: spans for every LLM call and tool call, logs vs metrics

Step 7 of 13

STEP 8

Approval Gates

Keeping a human in control of risky moments

1	Human-in-the-loop checkpoints before high-stakes or irreversible actions
2	Kill switches and manual override for a misbehaving run
3	Policy / rule engines for exactly what actions are allowed
4	Preventing infinite loops and runaway agent behavior

Step 8 of 13

STEP 9

Cost & Latency Controls

Making the same agent do more for less money, faster

1	Prompt caching — reusing repeated context cheaply instead of re-sending it
2	Token budgeting per request/session, and trimmed schemas/prompts
3	Caching LLM calls, parallel tool calls, async execution & task queues
4	Rate limiting and concurrency control across many agent runs

Step 9 of 13

STEP 10

Security

A new attack surface, specific to agents

1	Prompt injection: direct and indirect
2	Handling untrusted content (web pages, tool outputs, documents)
3	Tool permission scoping / least privilege — an agent only gets access it truly needs
4	Sandboxing tool execution (code execution, file access)

Step 10 of 13

STEP 11

Privacy

Handling real user data responsibly, not just correctly

1	Redacting PII before sending data to an LLM
2	Audit trails — recording exactly what the agent saw and did
3	Data retention policies — how long logs/context are kept and why
4	Compliance requirements for regulated data (health, finance, etc.)

Step 11 of 13

STEP 12

Content Management & Output Safety

Controlling what the agent produces, not just what it does

1	Grounding output in real data with RAG, instead of letting it hallucinate
2	Content moderation on generated output before it reaches a user
3	Confirmation steps before destructive or high-stakes actions
4	Versioning prompts and knowledge sources so content stays current and traceable

Step 12 of 13

STEP 13

Deployment & Reliability

Shipping changes safely and staying up when things go wrong

1	Retries with exponential backoff, per-step timeouts, circuit breakers
2	Model provider fallback — don't let one outage take the whole app down
3	Idempotency — a retried tool call should never double-charge or double-send
4	Regression testing (CI) and safe rollout: versioning, A/B testing, rollback

Step 13 of 13

Roadmap complete — 13 of 13 steps mapped.

These are exactly the pillars a production-grade agentic AI system needs — each one gets its own deep-dive session, same as the HLD journey: simple words, real examples, diagrams, and comprehension checks before moving on.